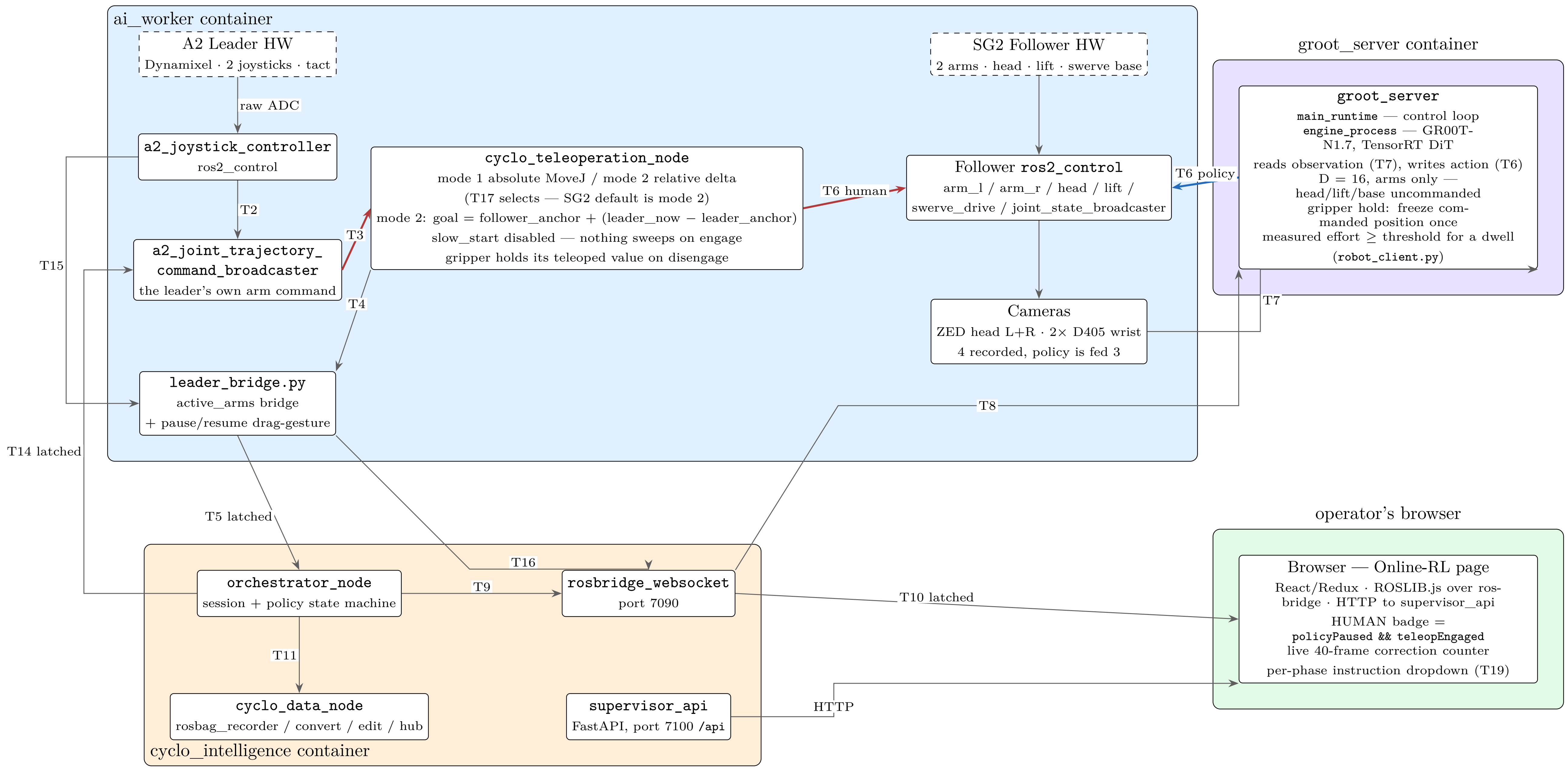
HG-DAgger / Online-RL — Runtime Control Path

FFW SG2 follower · A2 leader · GR00T-N1.7 policy

human commands the arms — policy commands the arms — status / feedback / control plane. Edge labels are IDs defined in the legend on p.3.



The operator's takeover loop

1. joystick drag-down ≥1 s → T16 → INFERENCING → PAUSED
2. tact press → T2 `COMMAND_TOGGLE` → that arm engages
3. tact press again → arm disengages, gripper holds its teleoped value
4. joystick drag-down ≥1 s → T16 → PAUSED → INFERENCING

The gaps 1→2 and 3→4 are PAUSED with no arm engaged: labelled −1, not human.

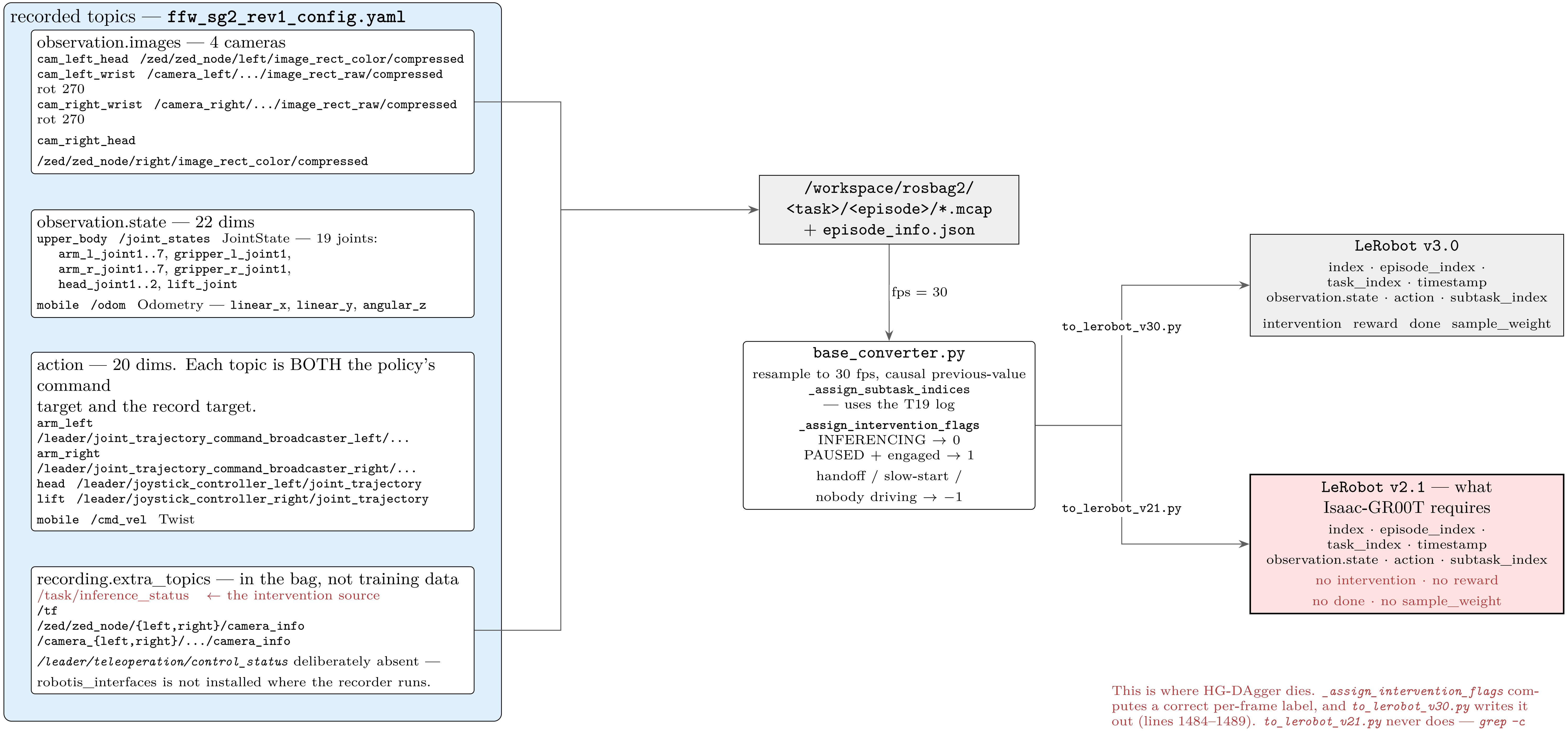
T16 is one topic with two meanings, resolved by the phase it arrives in.

T6 is ONE topic, written by BOTH sides. */leader/joint_trajectory_command_broadcaster_{left,right}/joint_trajectory*. Teleop node and policy publish to the same place, so the recorded *action* is correct in either mode — there is no stale-leader pose. T14 (*/task/policy_active*, latched) is what stops them contending.

T18 Home publishes straight from the browser into *ai_worker* over rosbridge, bypassing the orchestrator — the only UI action that does.

Recording and Conversion — what survives to training

Every recorded topic, and where the online-RL labels are lost



This is where HG-DAgger dies. *_assign_intervention_flags* computes a correct per-frame label, and *to_lerobot_v30.py* writes it out (lines 1484–1489). *to_lerobot_v21.py* never does — *grep -c intervention* on it returns 0. Isaac requires v2.1, so every GR00T export silently loses the human/policy label and degrades to plain behaviour cloning. Fix: *IMPROVEMENTS.md* §3.

Topic / Service Legend

Verified against `cyclo_intelligence_backup@main` and `aiw_backup@feature-a2`, 2026-08-30.

ID	Kind	Name	Type	Meaning / gotcha
T1	PUB/SUB	`/leader/joystick_controller/tact_trigger`	`std_msgs/String`	"left"/"right" tap. Record trigger disabled (`record_triggers_enabled: false`): one press both engages that leader arm and fires a record trigger, and engaging an arm during INFERENCING puts the leader onto T6 against the policy.
T2	PUB/SUB	`.../joystick_command`	`robotis_interfaces/TeleoperationCommand`	ENABLE / DISABLE / TOGGLE, per arm.
T3	PUB/SUB	`.../raw_joint_trajectory`	`trajectory_msgs/JointTrajectory`	The leader's own arm command, before mode handling.
T4	PUB/SUB	`/leader/teleoperation/control_status`	`robotis_interfaces/ControlModeStatus`	`active_arms` + mode. Not recorded — `robotis_interfaces` is absent where `service_bag_recorder` runs, and adding it killed the whole launch group after every takeover. Record T5 instead.
T5	PUB/SUB latched	`/leader/teleoperation/active_arms_str`	`std_msgs/String`	Plain-string translation of T4 from `leader_bridge.py`; loadable everywhere. The UI badge reads it. The bag should too — see IMPROVEMENTS §3.4.
T6	PUB/SUB	`/leader/joint_trajectory_command_broadcaster_{left,right}/joint_trajectory`	`trajectory_msgs/JointTrajectory`	Written by BOTH the teleop node and the policy. This is why `action` is valid in either mode. Contention is prevented by T14, not by using separate topics.
T7	PUB/SUB	`/joint_states`, `/odom`, 4 camera topics	`JointState`, `Odometry`, `CompressedImage`	Follower feedback and vision. 4 cameras are recorded; the policy's modality config declares 3.
T8	PUB/SUB	`/<backend>/engine_command`	`std_msgs/String`	Orchestrator → `groot_server`: load / unload / run.
T9	SERVICE	`/task/command`	`SendCommand.srv`	Start / Pause / Resume / Clear policy; Start / Stop / Cancel recording.
T10	PUB/SUB latched	`/task/inference_status`	`interfaces/InferenceStatus`	`inference_phase`: READY 0, LOADING 1, INFERENCING 2, PAUSED 3. Published on transitions only, so the converter must forward-fill. Sole source of the intervention label.
T11	SERVICE	`/data/recording`	`RecordingCommand.srv`	Recording lifecycle, orchestrator → `cyclo_data_node`.
T13	SERVICE	`/data/convert`, `/data/convert/status`	`StartConversion.srv`, `GetConversionStatus.srv`	LeRobot conversion. `convert_v21` and `convert_v30` both false means "run both".
T14	PUB/SUB latched	`/task/policy_active`	`std_msgs/Bool`	Mirrors T10's INFERENCING into `ai_worker`. Gates the leader chain off T6 so leader and policy never publish together.
T15	PUB/SUB	`.../sensorxel_joy_values`	`std_msgs/Float64MultiArray`	Raw joystick axes; `leader_bridge.py` watches them for the drag-hold gesture.
T16	PUB/SUB	`.../toggle_pause_request`	`std_msgs/Empty`	Fires once on a ≥1.0 s drag-hold. Lets the operator pause/resume the policy without the browser.
T17	PUB/SUB	`.../control_command`	`robotis_interfaces/ControlModeCommand`	Mode 1 absolute MoveJ, mode 2 relative delta. SG2 runs mode 2.
T18	PUB/SUB latched	`/task/home_in_progress`	`std_msgs/Bool`	One-shot Home trajectory. Published straight from the browser, bypassing the orchestrator — the only UI action that does.
T19	SERVICE	`/task/command` (UPDATE_INSTRUCTION)	`SendCommand.srv`	"Update Task Instruction". Appends to `instruction_change_log`, which splits the episode into correctly-timed sub-segments. Without it a continuous take yields one segment and every frame carries whatever instruction was set last.

Derived dataset columns

- **`observation.state`** (22) and **`action`** (20) — they differ: state carries `head_joint1/2`, action does not. Slice order follows YAML insertion order. The policy declares only the first 16 (both arms); the remainder are recorded but not trained on.
- **`task_index`** / **`subtask_index`** — per frame, from `episode_info.json`'s `segments`, split by `instruction_change_log` (T19). The episode's top-level `task_instruction` is a separate stable field; it is not the live per-phase instruction, which is overwritten on every T19 click.
- **`intervention`** — 1 = human drove, 0 = policy drove, −1 = excluded (handoff, leader slow-start, nobody driving, phase never observed). This is the inverse of `task_is_policy`, the name some downstream tooling expects; getting the polarity wrong raises nothing and trains on exactly the wrong frames. v3.0 only.
- **`reward`**, **`done`**, **`sample_weight`** — emitted alongside `intervention`. v3.0 only, so HIL-SERL-style training is also impossible from a v2.1 export.